Sri Lanka Institute of Information Technology



IT3091- Machine Learning
Year 3 Semester 1 – 2026

AMES HOUSING PRICE PREDICTION
Assignment
Final Report

Group ID	2026-A1-08
Campus/ Center name	Malabe
Specialization	Artificial Intelligence
Batch No	WE1.2

Group Members:

Name	Student IT Number
Pathirannehe K.P.J.M	IT24102348
Nandanayaka H.P.N.M	IT24101981
Dampahalage K.V.	IT24102444
Teshan Mathintha S.L.A.	IT24102362

CONTENTS

EXECUTIVE SUMMARY 5
1. PROJECT TEAM AND DIVISION OF RESPONSIBILITIES 6
2. END-TO-END SYSTEM ARCHITECTURE 7
2.1 Overview 7
2.2 Phase 1: Exploratory Data Analysis 7
2.3 Phase 2: Data Preprocessing and Feature Engineering 7
2.4 Phase 3: Model Training and Hyperparameter Tuning 8
2.5 Phase 4: Model Evaluation and Deployment 8
2.6 Overall Data Flow 10
3. EXPLORATORY DATA ANALYSIS (EDA) 11
3.1 Target Variable Analysis: SalePrice 11
3.2 Data Quality and Missingness Audit 11
3.3 Outlier and Anomaly Identification 12
3.4 Key Correlations and Relationships 12
4. DATA PREPROCESSING AND FEATURE ENGINEERING (V3) 14
4.1 Domain-Specific Feature Engineering 14
4.2 Data Anomaly Remediation 15
4.3 Feature Selection and Multicollinearity Pruning 15
4.4 Scikit-Learn ColumnTransformer Pipeline Architecture 17
5. COMPREHENSIVE DATASET AND DATA DICTIONARY 20
5.1 Pre-Encoding Feature Dictionary 20
5.2 Post-Encoding Feature Space Architecture 22
6. MODEL TRAINING AND HYPERPARAMETER TUNING 24
6.1 Algorithmic Paradigms and Hyperparameter Grids 24
6.2 Inverse Dollar Transformation Protocol 26
6.3 Diagnostic Plotting and Artifact Generation 26
7. MODEL EVALUATION AND COMPARATIVE ANALYSIS 28
7.1 Master Performance Comparison Table 28
7.2 Detailed Category Winners 29

7.3 Visual Comparison of Models 30
7.4 Feature Importance Analysis Across Models 31
7.5 Overfitting and Underfitting Analysis 32
7.6 Final Production Champion: XGBoost Regressor 34
8. CONSOLIDATED MASTER DECISION LOG 35
9. WEB APPLICATION AND INTERACTIVE DEPLOYMENT 37
9.1 Application Architecture 37
9.2 REST API Endpoints 37
9.3 Deployment Workflow 38
10. CODE WALKTHROUGH AND REPRODUCIBILITY GUIDE 40
10.1 Project Directory Structure 40
10.2 Reproducibility Step-by-Step Instructions 41
10.3 Reproducibility Considerations 44
11. CONCLUSION AND FUTURE WORK 45
11.1 Key Scientific Findings 45
11.2 Future Enhancements 46

Figures

Figure 1 : Overall Data Flow 10
Figure 2 : Application Interface 43

AMES HOUSING PRICE PREDICTION

Comprehensive University Final Project Report

Github link: https://github.com/VenuriDampahalage/ames-housing-price-prediction

Demo link: https://youtu.be/wh_skM2cbIM

EXECUTIVE SUMMARY

The objective of this project is to develop an enterprise-grade, accurate, and interpretable machine learning system to predict residential property sale prices in Ames, Iowa.

The project uses the Ames Housing Dataset, which contains 1,460 property sales and 81 attributes. The team completed an end-to-end machine learning lifecycle covering exploratory data analysis, data quality assessment, domain-specific feature engineering, data preprocessing, feature selection, multicollinearity reduction, hyperparameter optimization, model evaluation, and deployment through an interactive FastAPI web application.

Five regression algorithms were trained and evaluated using five-fold cross-validation and a strictly held-out test dataset containing 292 records.

The XGBoost Regressor was selected as the final champion model based on its overall performance across the evaluation metrics. It achieved the highest coefficient of determination, the lowest Root Mean Squared Error (RMSE), and the lowest Mean Squared Error (MSE) among the five evaluated models. Its test-set performance was:

- MAE : $16,884.70
- RMSE: $26,670.54
- MSE: $711,317,869.42 squared
- R^2 : 0.9073

The CatBoost Regressor also demonstrated strong predictive performance, achieving the lowest Mean Absolute Error (MAE) of $16,528.23 and a reported log-scale MSE of 0.0206. However, XGBoost achieved the best overall average rank across MAE, RMSE, dollar-scale MSE, and R^2, with an average rank of 1.25, compared with 1.75 for CatBoost.

The final system provides an interactive web interface that accepts property attributes and returns a predicted sale price. The project demonstrates a complete machine learning workflow, from raw data analysis to trained model deployment.

1. PROJECT TEAM AND DIVISION OF RESPONSIBILITIES

To ensure academic rigor and modular engineering, the project was divided into four phases. Each phase was led by a designated team member.

Member	Full Name and GitHub Identity	Primary Responsibility	Core Deliverables
Member 1	Venuri Dampahalage (VenuriDampahalage)	Exploratory Data Analysis (EDA)	eda.ipynb, 26-point EDA insight log, data quality audit, univariate, bivariate and multivariate analysis
Member 2	Jayashan Manodya (JayashanManodya)	Data Preprocessing and Feature Engineering (v3)	preprocessV3.ipynb, preprocessing pipelines, 16 engineered features, multicollinearity and association pruning, data-new exports
Member 3	Teshan (teshanDev / mathinthateshan)	Model Training and Hyperparameter Tuning	Model_Training.ipynb, five-fold RandomizedSearchCV, five model pipelines, trained models, predictions and diagnostic plots
Member 4	Nadeesha Malshan (NadeeshaMalshan)	Model Evaluation, Benchmarking and Web Deployment	Model_Evaluation.ipynb, metric comparison framework, champion model selection and FastAPI web interface

2. END-TO-END SYSTEM ARCHITECTURE

2.1 Overview

The system follows a sequential machine learning workflow divided into four major phases.

1. **Exploratory Data Analysis** - Member 1: Venuri Dampahalage.
2. **Data Preprocessing and Feature Engineering** - Member 2: Jayashan Manodya.
3. **Model Training and Hyperparameter Tuning** - Member 3: Teshan.
4. **Model Evaluation and Deployment** - Member 4: Nadeesha Malshan.

The output of each phase becomes the input to the next phase. This creates a structured workflow from the raw dataset to the final deployed prediction application.

2.2 Phase 1: Exploratory Data Analysis

Responsible member: Venuri Dampahalage.

Input: Raw dataset containing 1,460 rows and 81 columns.

Procedure:

1. Load the raw dataset from dataset.csv.
2. Use eda.ipynb to inspect individual feature distributions and relationships between variables.
3. Perform univariate and bivariate analysis.
4. Examine missing values and distinguish structural missingness from genuine missing data.
5. Investigate outliers and unusual property records.
6. Identify correlations between property characteristics and sale prices.
7. Produce a 26-point EDA insight log and hypotheses for later preprocessing.

Output: Data quality findings, correlation insights, missingness analysis, outlier findings and documented hypotheses.

2.3 Phase 2: Data Preprocessing and Feature Engineering

Responsible member: Jayashan Manodya.

Input: Findings from Phase 1.

Procedure:

1. Create 16 domain-specific engineered features.
2. Select numerical and categorical predictors using Pearson correlation and mutual information.
3. Reduce redundant features using correlation and categorical association analysis.
4. Retain 24 selected predictors and the SalePrice target.

5. Split the data into training and testing partitions using an 80/20 split.
6. Fit a Scikit-Learn ColumnTransformer pipeline using only the training data.
7. Apply StandardScaler to 11 numerical features.
8. Apply OrdinalEncoder to seven ordinal features.
9. Apply OneHotEncoder to six nominal features, producing 73 indicator columns.
10. Transform the target variable using the natural logarithm of one plus SalePrice.
11. Export the processed training and testing datasets.

Output: Leakage-free processed datasets with 91 predictor columns, together with the transformed target values.

2.4 Phase 3: Model Training and Hyperparameter Tuning

Responsible member: Teshan.

Input: Processed training data from Phase 2.

Procedure:

1. Use five-fold RandomizedSearchCV to search for suitable hyperparameters.
2. Optimize negative mean squared error in the log-transformed target domain.
3. Train five regression algorithms:
    - Ridge Regression.
    - Random Forest Regressor.
    - XGBoost Regressor.
    - LightGBM Regressor.
    - CatBoost Regressor.
4. Save the trained model artifacts.
5. Generate test predictions for each model.
6. Produce actual-versus-predicted plots and residual distribution plots.
7. Export the models, predictions and visualizations for the evaluation phase.

Output: Five trained regression models, their prediction files and diagnostic visualizations.

2.5 Phase 4: Model Evaluation and Deployment

Responsible member: Nadeesha Malshan.

Input: Trained models and prediction artifacts from Phase 3.

Procedure:

1. Convert log-scale predictions back into dollar values using the inverse transformation.
2. Calculate MAE, RMSE, log-scale MSE and R^2.

3. Compare all five models using multiple evaluation metrics.
4. Select XGBoost as the production champion model.
5. Save the selected model and preprocessing artifacts.
6. Integrate the model into an interactive FastAPI web application.
7. Provide endpoints for property valuation, example property presets, application health checks and feature summaries.

Output: An evaluated champion model and a web application for real-time property valuation.

2.6 Overall Data Flow

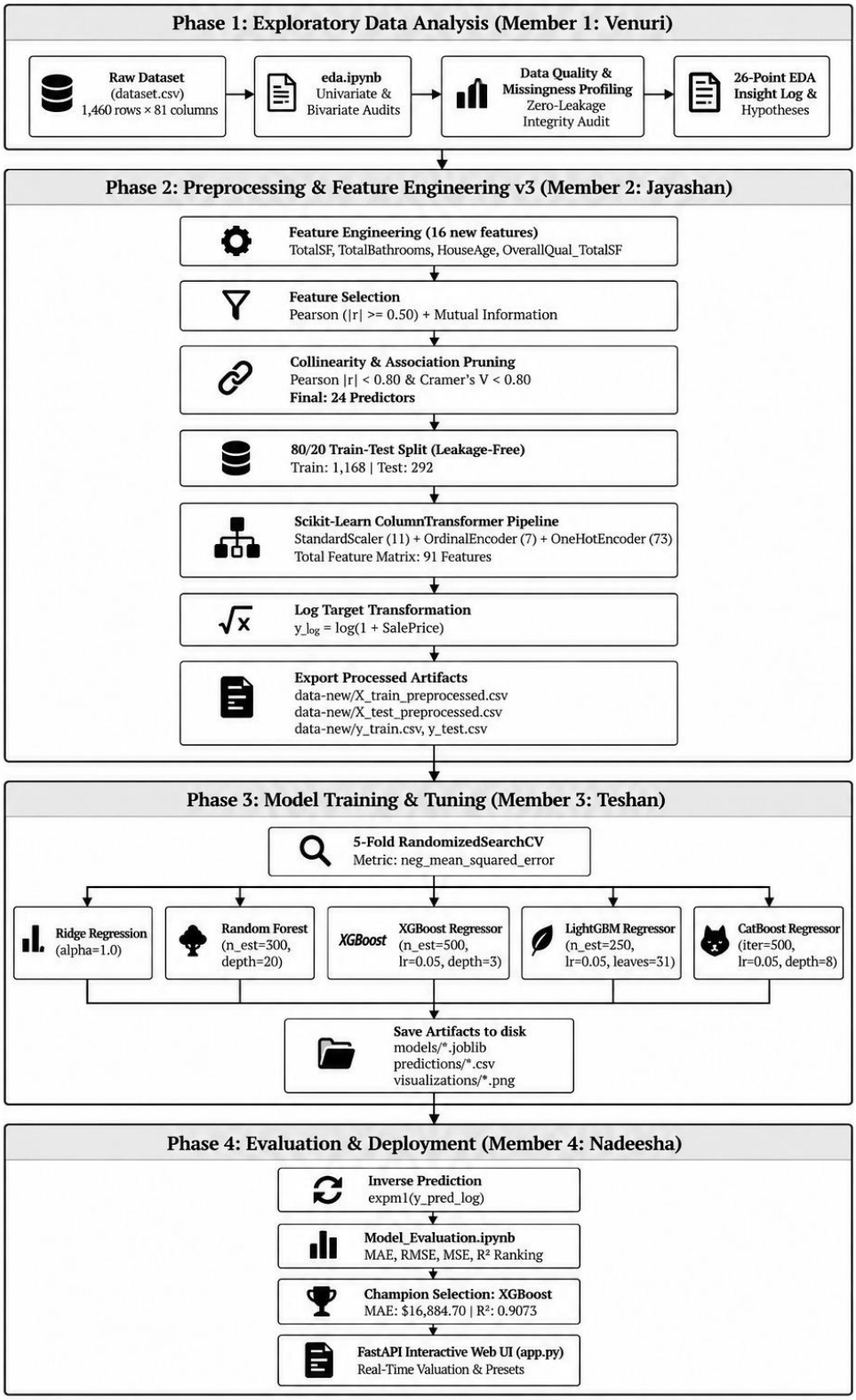


Figure 1 : Overall Data Flow

3. EXPLORATORY DATA ANALYSIS (EDA)

Phase leader: Member 1 - Venuri Dampahalage.

The exploratory analysis was conducted using eda.ipynb. It provided the empirical foundation for the later preprocessing and model training stages.

3.1 Target Variable Analysis: SalePrice

Distribution characteristics

The raw SalePrice variable exhibited a positively skewed distribution with a long right tail.

- Skewness: 1.88.
- Kurtosis: 6.54.
- Mean sale price: $180,921.20.
- Median sale price: $163,000.00.
- Standard deviation: $79,442.50.

The positive skew indicates that most properties were concentrated in lower or middle price ranges, while a smaller number of expensive properties created a long upper tail.

Log transformation

The project applied the following transformation:

SalePrice_log = ln(1 + SalePrice)

The transformation reduced the reported skewness to 0.12.

The purpose was to reduce the influence of extreme prices, stabilize the variance and make model optimization less dominated by expensive properties. It also supported evaluation in a log-transformed domain, where proportional differences in prices receive greater consideration.

3.2 Data Quality and Missingness Audit

The analysis examined all 81 features to distinguish between structural missingness and genuine missing data.

A. Structural missingness

In this dataset, some missing values represent the absence of a physical property feature rather than an unknown value.

Feature	Approximate Missing Percentage	Interpretation
PoolQC	99.52%	The property does not have a pool.
MiscFeature	96.30%	The property lacks a miscellaneous feature, such as a tennis court or elevator.
Alley	93.77%	The property lacks alley access.

Feature	Approximate Missing Percentage	Interpretation
Fence	80.75%	The property does not have a fence enclosure.
FireplaceQu	47.26%	The property has no fireplace.
GarageType, GarageFinish, GarageQual and GarageCond	Approximately 5.5%	The property lacks a garage.
BsmtQual, BsmtCond, BsmtExposure, BsmtFinType1 and BsmtFinType2	Approximately 2.5%	The property lacks a basement.

Treating these values as ordinary unknown values could remove useful information about the property. Their interpretation must therefore reflect the meaning of the dataset.

B. Genuine missingness

Feature	Missingness	Treatment or Interpretation
LotFrontage	17.74%	Missing street frontage measurements were associated with parcel size and lot configuration.
MasVnrType and MasVnrArea	0.55%	Imputed as None and 0, respectively.
Electrical	One record	Imputed using the dominant category, SBrkr.

3.3 Outlier and Anomaly Identification

Two notable properties in the Edwards neighborhood had living areas exceeding 4,000 square feet but sold for less than $200,000.

These records could exert substantial influence on regression models, particularly models that minimize squared errors. They may represent partial sales or unusual commercial-like parcels.

The project retained these observations and used a log transformation to reduce the effect of extreme values instead of automatically removing them.

Data leakage safeguard

The EDA process established a rule that statistical properties and global preprocessing parameters from the test set should not be used to inform training decisions. This supported the project's objective of keeping the training and testing processes separate.

3.4 Key Correlations and Relationships

A. Dominant continuous predictors

Feature	Pearson Correlation with SalePrice
GrLivArea	0.709
TotalBsmtSF	0.614
1stFlrSF	0.606

Feature	Pearson Correlation with SalePrice
GarageCars	0.640

These results show that living area, basement area, first-floor area and garage capacity have meaningful relationships with sale prices.

B. Dominant ordinal and categorical predictors

- OverallQual: Pearson correlation of 0.791.
- Neighborhood: Median prices ranged from approximately $88,000 in Meadow Village to $315,000 in Northridge Heights.
- ExterQual: Exterior material quality was identified as an important predictor.
- KitchenQual: Kitchen quality was also identified as an important predictor.

C. Severe multicollinearity indicators

The analysis identified the following highly correlated feature pairs:

Feature Pair	Correlation
GarageCars and GarageArea	0.882
YearBuilt and GarageYrBlt	0.826
GrLivArea and TotRmsAbvGrd	0.825
1stFlrSF and TotalBsmtSF	0.820

These relationships were considered during feature selection and redundancy pruning.

4. DATA PREPROCESSING AND FEATURE ENGINEERING (V3)

Phase leader: Member 2 - Jayashan Manodya.

The preprocessing pipeline was implemented in preprocessV3.ipynb. It transformed the raw dataset into a numerical feature matrix suitable for machine learning.

4.1 Domain-Specific Feature Engineering

Sixteen engineered features were created to represent property characteristics and interactions more directly.

No.	Engineered Feature	Mathematical Definition	Purpose
1	TotalSF	TotalBsmtSF + 1stFlrSF + 2ndFlrSF	Measures total enclosed finished living space across the basement and floors.
2	TotalPorchSF	OpenPorchSF + EnclosedPorch + 3SsnPorch + ScreenPorch	Combines outdoor leisure areas into one measurement.
3	TotalBathrooms	FullBath + 0.5 × HalfBath + BsmtFullBath + 0.5 × BsmtHalfBath	Calculates the effective number of bathrooms.
4	HouseAge	YrSold − YearBuilt	Measures the property's age at the time of sale.
5	RemodAge	YrSold − YearRemodAdd	Measures the number of years since the recorded remodeling year.
6	TotalFinishedBsmtSF	BsmtFinSF1 + BsmtFinSF2	Measures finished basement space.
7	TotalLivingSF	GrLivArea + TotalFinishedBsmtSF	Combines above-grade living area and finished basement area.
8	HasGarage	I(GarageArea > 0)	Indicates whether the property has a garage.
9	HasBasement	I(TotalBsmtSF > 0)	Indicates whether the property has a basement.
10	HasFireplace	I(Fireplaces > 0)	Indicates whether the property has a fireplace.
11	HasPool	I(PoolArea > 0)	Indicates whether the property has a pool.
12	GarageAge	YrSold − GarageYrBlt	Measures the age of the garage at the time of sale.
13	OverallQual_GrLivArea	OverallQual × GrLivArea	Captures the interaction between property quality and above-grade living area.
14	OverallQual_TotalSF	OverallQual × TotalSF	Captures the interaction between overall quality and total enclosed space.
15	OverallCond_HouseAge	OverallCond / (HouseAge + 1)	Combines overall condition with the age of the property.

No.	Engineered Feature	Mathematical Definition	Purpose
16	OutdoorArea	LotArea − (1stFlrSF + GarageArea)	Estimates remaining lot area after subtracting selected building footprints; later dropped because it introduced noise.

The strongest reported engineered feature was OverallQual_TotalSF, with a Pearson correlation of approximately 0.88 with SalePrice.

This feature combines construction quality and total space, allowing the model to represent the possibility that additional square footage has a different price effect depending on the property's quality.

4.2 Data Anomaly Remediation

Negative RemodAge anomaly

One record had a remodeling completion year recorded one year after the contract sale year:

YrSold < YearRemodAdd

This produced a negative RemodAge value of -1.

The project applied lower-bound clipping:

RemodAge = max(0, RemodAge)

This prevents a negative remodeling age from being used as a normal elapsed-time measurement.

OutdoorArea removal

OutdoorArea was removed after statistical auditing. The variation in LotArea introduced substantial noise when building footprints were subtracted from it.

The feature was therefore not retained in the final selected feature set.

4.3 Feature Selection and Multicollinearity Pruning

The project used a two-stage selection process.

A. Numerical selection using Pearson correlation

The numerical feature selection threshold was:

$|r| \geq 0.50$

The principal selected numerical candidates were:

Feature	Pearson Correlation
OverallQual_TotalSF	0.88
OverallQual	0.79
TotalSF	0.78

Feature	Pearson Correlation
GrLivArea	0.71
GarageCars	0.64
TotalBathrooms	0.63
TotalBsmtSF	0.61
1stFlrSF	0.61
FullBath	0.56
TotRmsAbvGrd	0.53
HouseAge	-0.52

B. Categorical selection using mutual information

The categorical selection threshold was:

Mutual Information (MI) ≥ 0.10

The principal selected categorical features were:

Feature	Mutual Information
Neighborhood	0.54
BsmtQual	0.32
KitchenQual	0.32
ExterQual	0.32
GarageFinish	0.27
FireplaceQu	0.21
HeatingQC	0.17
Foundation	0.17
GarageType	0.15
Exterior2nd	0.14
MSSubClass	0.14
BsmtFinType1	0.13
MasVnrType	0.10

C. Numerical redundancy pruning

Numerical feature pairs with an absolute Pearson correlation of 0.80 or greater were inspected.

- GarageArea was dropped in favor of GarageCars because GarageCars provided a more stable discrete measure and had a strong relationship with SalePrice.
- 1stFlrSF and TotalBsmtSF were retained alongside TotalSF because they represented useful component areas as well as contributing to the engineered total-space feature.

D. Categorical association pruning

Categorical pairs were examined using Cramér's V, with 0.80 used as the association threshold.

Exterior1st was removed in favor of Exterior2nd, which had higher mutual information with the target in the reported selection process.

Final selected feature set

The final dataset contained **24 selected predictors plus the SalePrice target variable**.

4.4 Scikit-Learn ColumnTransformer Pipeline Architecture

The preprocessing pipeline was fitted strictly on the training data and then applied to the test data.

- Training partition: 1,168 records.
- Testing partition: 292 records.
- Training proportion: 80%.
- Testing proportion: 20%.

A. Numerical pipeline: StandardScaler

Eleven numerical features were standardized using the z-score formula:

$$z = (x - \mu) / \sigma$$

Where:

- x is the original feature value.
- μ is the mean calculated from the training data.
- σ is the standard deviation calculated from the training data.
- z is the standardized value.

The numerical features were:

1. OverallQual_TotalSF
2. OverallQual
3. TotalSF
4. GrLivArea
5. GarageCars
6. TotalBathrooms
7. TotalBsmtSF
8. 1stFlrSF
9. FullBath
10. TotRmsAbvGrd
11. HouseAge

B. Ordinal pipeline: OrdinalEncoder

Seven ordinal features were encoded as ordered integer ranks.

The features were:

- BsmtQual
- KitchenQual
- ExterQual
- GarageFinish
- FireplaceQu
- HeatingQC
- BsmtFinType1

The reported category orders were:

BsmtQual, KitchenQual, ExterQual, FireplaceQu and HeatingQC

None → Po → Fa → TA → Gd → Ex

GarageFinish

None → Unf → RFn → Fin

BsmtFinType1

None → Unf → LwQ → Rec → BLQ → ALQ → GLQ

This encoding preserves the meaningful order of the categories rather than treating every category as unrelated.

C. Nominal pipeline: OneHotEncoder

Six nominal features were one-hot encoded.

Feature	Number of Indicator Columns
Neighborhood	25
MSSubClass	15
Exterior2nd	16
GarageType	7
Foundation	6
MasVnrType	4
Total	**73**

The OneHotEncoder used handle_unknown='ignore' so that previously unseen categories could be handled without producing an ordinary unknown-category encoding error.

D. Target transformation

The target was transformed separately:

y_train_log = log(1 + y_train)

y_test_log = log(1 + y_test)

The logarithmic transformation was used to reduce the influence of the target's right-skewed distribution during model training.

E. Final processed feature space

The final processed matrix contained 91 columns:

- 11 standardized numerical features.
- 7 ordinal-encoded features.
- 73 one-hot-encoded indicator columns.

Total: 11 + 7 + 73 = 91 processed features.

This produced the numerical input matrix used by the regression models.

5. COMPREHENSIVE DATASET AND DATA DICTIONARY

5.1 Pre-Encoding Feature Dictionary

The final feature-selection process retained 24 predictors and the SalePrice target variable.

The following table describes each selected predictor, its origin, data type, physical meaning and reported predictive metric.

No.	Feature Name	Origin	Category	Type	Physical Meaning	Predictive Metric
1	OverallQual_TotalSF	Engineered	Numerical	Continuous	OverallQual × TotalSF; represents the interaction between quality and total enclosed space.	Pearson r = 0.88
2	OverallQual	Raw	Numerical	Discrete	Overall material and finish quality, rated from 1 (Very Poor) to 10 (Very Excellent).	Pearson r = 0.79
3	TotalSF	Engineered	Numerical	Continuous	TotalBsmtSF + 1stFlrSF + 2ndFlrSF; total enclosed space across the basement and floors.	Pearson r = 0.78
4	GrLivArea	Raw	Numerical	Continuous	Finished living area above ground level, including relevant upper floors, measured in square feet.	Pearson r = 0.71
5	GarageCars	Raw	Numerical	Discrete	Garage parking capacity, from zero to four vehicles in the dataset description.	Pearson r = 0.64
6	TotalBathrooms	Engineered	Numerical	Continuous	Effective bathroom count calculated using full and half bathrooms.	Pearson r = 0.63
7	TotalBsmtSF	Raw	Numerical	Continuous	Total basement area, including finished and unfinished space.	Pearson r = 0.61
8	1stFlrSF	Raw	Numerical	Continuous	Finished first-floor area in square feet.	Pearson r = 0.61
9	FullBath	Raw	Numerical	Discrete	Number of full bathrooms above ground level.	Pearson r = 0.56

No.	Feature Name	Origin	Category	Type	Physical Meaning	Predictive Metric
10	TotRmsAbvGrd	Raw	Numerical	Discrete	Total rooms above ground level, excluding bathrooms and closets.	Pearson r = 0.53
11	HouseAge	Engineered	Numerical	Discrete	YrSold − YearBuilt, measured in years.	Pearson r = -0.52
12	BsmtQual	Raw	Ordinal	Ranked	Basement height and structural quality.	MI = 0.32
13	KitchenQual	Raw	Ordinal	Ranked	Quality of kitchen materials, appliances and layout.	MI = 0.32
14	ExterQual	Raw	Ordinal	Ranked	Quality of exterior construction materials and siding.	MI = 0.32
15	GarageFinish	Raw	Ordinal	Ranked	Garage finish status: None, Unfinished, Rough or Finished.	MI = 0.27
16	FireplaceQu	Raw	Ordinal	Ranked	Fireplace quality and condition.	MI = 0.21
17	HeatingQC	Raw	Ordinal	Ranked	Heating system quality and condition.	MI = 0.17
18	BsmtFinType1	Raw	Ordinal	Ranked	Quality category of the first basement finished area.	MI = 0.13
19	Neighborhood	Raw	Nominal	Categorical	The neighborhood in which the property is located; 25 categories were reported.	MI = 0.54
20	MSSubClass	Raw	Nominal	Categorical	Type of dwelling involved in the sale.	MI = 0.14
21	Exterior2nd	Raw	Nominal	Categorical	Secondary exterior wall covering material.	MI = 0.14
22	GarageType	Raw	Nominal	Categorical	Garage arrangement, such as attached, detached or built-in.	MI = 0.15
23	Foundation	Raw	Nominal	Categorical	Foundation type, such as poured concrete, cinder block or slab.	MI = 0.17
24	MasVnrType	Raw	Nominal	Categorical	Masonry veneer type, such as brick face, stone or none.	MI = 0.10

No.	Feature Name	Origin	Category	Type	Physical Meaning	Predictive Metric
Target	SalePrice	Raw	Target	Continuous	Actual residential property sale price in US dollars.	Skewness = 1.88; mean = $180,921

Note: The table preserves the predictive metrics reported in the source project document. Feature-selection rankings and correlation values should be interpreted according to the calculations used in the project.

5.2 Post-Encoding Feature Space Architecture

After preprocessing, the model input consisted of 91 columns.

A. Numerical features: 11 columns

These features were standardized using StandardScaler.

1. OverallQual_TotalSF
2. OverallQual
3. TotalSF
4. GrLivArea
5. GarageCars
6. TotalBathrooms
7. TotalBsmtSF
8. 1stFlrSF
9. FullBath
10. TotRmsAbvGrd
11. HouseAge

B. Ordinal features: 7 columns

These features were encoded using OrdinalEncoder:

1. BsmtQual
2. KitchenQual
3. ExterQual
4. GarageFinish
5. FireplaceQu
6. HeatingQC
7. BsmtFinType1

The category ranks preserve the order of quality or completion levels.

C. Nominal features: 73 columns

OneHotEncoder transformed the six nominal features into 73 binary indicator columns.

Neighborhood - 25 columns:

Blmngtn, Blueste, BrDale, BrkSide, ClearCr, CollgCr, Crawfor, Edwards, Gilbert, IDOTRR, MeadowV, Mitchel, NAmes, NPkVill, NWAmes, NoRidge, NridgHt, OldTown, SWISU, Sawyer, SawyerW, Somerst, StoneBr, Timber and Veenker.

MSSubClass - 15 columns:

20, 30, 40, 45, 50, 60, 70, 75, 80, 85, 90, 120, 160, 180 and 190.

Exterior2nd - 16 columns:

AsbShng, AsphShn, Brk Cmn, BrkFace, CBlock, CmentBd, HdBoard, ImStucc, MetalSd, Other, Plywood, Stone, Stucco, VinylSd, Wd Sdng and Wd Shng.

GarageType - 7 columns:

2Types, Attchd, Basment, BuiltIn, CarPort, Detchd and None.

Foundation - 6 columns:

BrkTil, CBlock, PConc, Slab, Stone and Wood.

MasVnrType - 4 columns:

BrkCmn, BrkFace, None and Stone.

The complete processed feature space therefore contained 91 columns: 11 numerical, 7 ordinal and 73 nominal indicator columns.

6. MODEL TRAINING AND HYPERPARAMETER TUNING

Phase leader: Member 3 - Teshan.

Model training was implemented in Model_Training.ipynb. Five regression algorithms were trained and tuned using five-fold cross-validation with RandomizedSearchCV.

The optimization metric was negative mean squared error (neg_mean_squared_error) in the log-transformed target domain.

6.1 Algorithmic Paradigms and Hyperparameter Grids

A. Ridge Regression

Model family: Regularized linear regression using an L2 penalty.

Ridge Regression adds a penalty to large model coefficients. This helps control model complexity and can reduce the effect of multicollinearity.

Hyperparameter search:

- alpha: 0.1, 1.0, 10.0, 100.0, 500.0.

Selected hyperparameter:

- alpha = 1.0.

B. Random Forest Regressor

Model family: Bagging ensemble of decision trees.

Random Forest combines predictions from multiple decision trees to capture nonlinear relationships and reduce reliance on a single tree.

Hyperparameter search:

- n_estimators: 100, 200, 300.
- max_depth: 10, 20, None.
- min_samples_split: 2, 5, 10.

Selected hyperparameters:

- n_estimators = 300.
- max_depth = 20.
- min_samples_split = 2.

C. XGBoost Regressor

Model family: Extreme Gradient Boosting.

XGBoost builds trees sequentially, with each new tree improving the existing ensemble's predictions.

Hyperparameter search:

- n_estimators: 100, 250, 500.
- learning_rate: 0.01, 0.05, 0.1.
- max_depth: 3, 5, 7.

Selected hyperparameters:

- n_estimators = 500.
- learning_rate = 0.05.
- max_depth = 3.

D. LightGBM Regressor

Model family: Leaf-wise gradient boosting.

LightGBM uses a histogram-based approach and grows trees by selecting leaf splits that offer strong improvements to the objective.

Hyperparameter search:

- n_estimators: 100, 250, 500.
- learning_rate: 0.01, 0.05, 0.1.
- num_leaves: 31, 50, 100.

Selected hyperparameters:

- n_estimators = 250.
- learning_rate = 0.05.
- num_leaves = 31.

E. CatBoost Regressor

Model family: Symmetric or oblivious gradient boosting.

CatBoost uses symmetric decision trees, where the same splitting condition is applied across a tree level.

Hyperparameter search:

- iterations: 200, 500.
- learning_rate: 0.01, 0.05, 0.1.
- depth: 4, 6, 8.

Selected hyperparameters:

- iterations = 500.
- learning_rate = 0.05.
- depth = 8.

6.2 Inverse Dollar Transformation Protocol

The models were trained using the log-transformed target:

y_log = ln(1 + SalePrice)

Therefore, the predictions initially produced by the models were in the log domain rather than in US dollars.

To convert predictions back to the original price scale, the project used the inverse transformation:

Predicted SalePrice = exp(y_pred_log) − 1

In Python, this is implemented as:

np.expm1(y_pred_log)

The actual test target values were converted back in the same way:

y_test_dollars = np.expm1(y_test_log)

This allowed the team to calculate evaluation metrics such as MAE and RMSE in actual dollar values.

6.3 Diagnostic Plotting and Artifact Generation

The training phase generated diagnostic plots for each of the five models.

A. Actual-versus-predicted scatter plot

This plot compares actual property sale prices with predicted property sale prices.

A reference identity line represents perfect predictions:

y = y_pred

Points closer to the reference line indicate predictions closer to the actual values.

B. Residual distribution histogram with KDE

Residuals were calculated as:

Residual = Actual Price − Predicted Price

The histogram and kernel density estimate were used to inspect the distribution of prediction errors and assess whether they were centered near zero.

C. Artifact persistence

The training phase exported the following artifacts:

- Trained model files in models/*.joblib.
- Test prediction files in predictions/*.csv.
- Diagnostic figures in visualizations/*.png.
- A handoff_package.zip file containing artifacts for the evaluation phase.

These outputs allowed the evaluation phase to compare the models without repeating the entire training process.

7. MODEL EVALUATION AND COMPARATIVE ANALYSIS

Phase leader: Member 4 - Nadeesha Malshan.

Model evaluation was conducted using Model_Evaluation.ipynb. Five regression models were evaluated on the held-out test dataset of 292 houses, representing 20% of the original 1,460 observations. The remaining 1,168 houses were used for training.

7.1 Master Performance Comparison Table

Model Rank	Algorithm	MAE ($)	RMSE ($)	MSE	R^2 Score	Overall Rank Score	Main Result
1	XGBoost Regressor	16,884.70	26,670.54	711,317,869.42	0.9073	**1.25**	Highest R^2, lowest RMSE and MSE
2	CatBoost Regressor	16,528.23	27,116.86	735,324,002.30	0.9041	**1.75**	Lowest MAE and log-scale MSE
3	Random Forest Regressor	17,682.37	30,886.44	953,972,227.89	0.8756	**3.00**	Strong nonlinear baseline, with overfitting
4	Ridge Regression	18,759.94	31,905.66	1,017,971,039.49	0.8673	**4.25**	Linear benchmark, with underfitting
5	LightGBM Regressor	18,245.00	32,896.85	1,082,202,979.55	0.8589	**4.75**	Fast histogram-based boosting, with overfitting

Ranking note: The overall rank score was calculated by averaging the ordinal ranks across MAE, RMSE, MSE, and R^2. For error metrics, lower values received better ranks; for R^2, higher values received better ranks.

XGBoost achieved an overall rank score of 1.25, while CatBoost achieved 1.75. Therefore, XGBoost was selected as the production champion because it delivered the strongest combined performance across the principal evaluation metrics and demonstrated the best generalization among the nonlinear models.

7.2 Detailed Category Winners

A. Lowest Mean Absolute Error: CatBoost

CatBoost achieved an MAE of **$16,528.23**, the lowest among the five evaluated models.

MAE measures the average absolute difference between actual and predicted sale prices. Its value is expressed in dollars, making it particularly useful for communicating the typical magnitude of valuation errors.

Compared with the dataset's reported average sale price of $180,921.20, CatBoost's MAE represents approximately 9.13%.

Although CatBoost was the strongest model under this metric, its advantage in average absolute error did not outweigh XGBoost's combined performance across the other evaluation criteria.

B. Lowest Root Mean Squared Error: XGBoost

XGBoost achieved an RMSE of **$26,670.54**, outperforming CatBoost's $27,116.86 by approximately $446.32.

RMSE penalizes larger prediction errors more heavily because the errors are squared before averaging. Consequently, XGBoost's lower RMSE indicates better performance under a metric that gives greater weight to substantial valuation mistakes.

However, log-scale MSE and dollar-scale RMSE measure errors in different domains. They should therefore be interpreted separately rather than treated as interchangeable metrics.

C. Highest Coefficient of Determination: XGBoost

XGBoost achieved the highest test-set R^2 score of **0.9073**, followed by CatBoost at 0.9041.

R^2 measures how much of the variation in the target variable is explained by a model relative to predicting the mean sale price.

The results indicate that XGBoost explained approximately 90.73% of the observed variation in sale prices within the held-out test set, while CatBoost explained approximately 90.41%.

D. Lowest Dollar-Scale MSE: XGBoost

XGBoost achieved the lowest reported dollar-scale MSE of **$711,317,869.42**, followed by CatBoost at $735,324,002.30.

Since MSE squares the prediction errors, it places greater emphasis on larger mistakes. XGBoost's lower value supports its selection when evaluating the squared dollar-scale error of property valuations.

E. Best Generalization Among Nonlinear Models: XGBoost

The train-versus-test evaluation identified XGBoost as the best-generalizing nonlinear model.

Its training R^2 was 0.9644, compared with a test R^2 of 0.9073, resulting in a generalization gap of **5.71 percentage points**. This was smaller than CatBoost's 7.75-point gap, Random Forest's 10.17-point gap, and LightGBM's 11.03-point gap.

This result supported the final production selection because a model should not only perform well on training data but also maintain strong predictive performance on unseen properties.

7.3 Visual Comparison of Models

The evaluation visualizations compared the models using their R^2, RMSE and MAE values.

R^2 comparison

- XGBoost: 0.9073
- CatBoost: 0.9041
- Random Forest: 0.8756
- Ridge Regression: 0.8673
- LightGBM: 0.858

XGBoost and CatBoost formed the leading group, with both achieving R^2 values above 0.90. XGBoost achieved the highest score.

RMSE comparison

- **XGBoost: $26,670.54**
- **CatBoost: $27,116.86**
- **Random Forest: $30,886.44**
- **Ridge Regression: $31,905.66**
- **LightGBM: $32,896.85**

XGBoost recorded the lowest RMSE, followed by CatBoost. Both models substantially outperformed the other three models under this metric.

MAE comparison

- CatBoost: $16,528.23
- XGBoost: $16,884.70
- Random Forest: $17,682.37
- LightGBM: $18,245.00

- Ridge Regression: $18,759.94

MSE comparison

- XGBoost: $711,317,869.42
- CatBoost: $735,324,002.30
- Random Forest: $953,972,227.89
- Ridge Regression: $1,017,971,039.49
- LightGBM: $1,082,202,979.55

XGBoost achieved the lowest dollar-scale MSE. This further supported its selection for the production application.

7.4 Feature Importance Analysis Across Models

The evaluation identified several features that consistently contributed to property price prediction.

A. OverallQual_TotalSF

OverallQual_TotalSF was the strongest reported engineered predictor.

The source document reported that it accounted for approximately 79.74% of total split importance in Random Forest and was a dominant feature in Ridge, XGBoost and CatBoost.

B. HouseAge

HouseAge was identified as another influential feature.

It represents the age of a property at the time of sale and captures the relationship between property age and sale price.

C. TotalSF and GrLivArea

These features represent physical living space. TotalSF combines basement and floor areas, while GrLivArea represents above-grade finished living area.

D. TotalBathrooms and GarageCars

These features represent important functional characteristics of a property. They help describe the property's facilities and capacity.

E. Neighborhood

Neighborhood indicators, including StoneBr, NridgHt and Crawfor, were identified as influential.

The source reported positive Ridge coefficients of approximately +0.1507 and +0.1163 for selected neighborhood indicators, as well as frequent tree splits for neighborhood-related features in CatBoost.

Feature-importance values are model-dependent and should be interpreted according to the method used to calculate them.

7.5 Overfitting and Underfitting Analysis

The project compared training and testing performance to determine whether each model generalized effectively to unseen data.

Model	Train R^2	Test R^2	Generalization Gap	Train MAE ($)	Test MAE ($)	Diagnosis
XGBoost	0.9644	0.9073	5.71 pp	10,110.66	16,884.70	Best-balanced nonlinear model
CatBoost	0.9816	0.9041	7.75 pp	7,402.46	16,528.23	Good generalization, mild variance
Ridge Regression	0.8724	0.8673	0.52 pp	16,781.24	18,759.94	Underfitting, high bias
Random Forest	0.9773	0.8756	10.17 pp	6,690.67	17,682.37	Overfitting, high variance
LightGBM	0.9693	0.8589	11.03 pp	7,138.08	18,245.00	Overfitting, high variance

A. XGBoost: Best Generalization

XGBoost achieved a training R^2 of 0.9644 and a test R^2 of 0.9073, producing a 5.71-percentage-point generalization gap.

Its documented configuration included a maximum tree depth of 3 and a learning rate of 0.05. These settings, together with the model's regularization, contributed to a balance between learning meaningful patterns and limiting the effect of training-specific noise.

B. CatBoost: Strong Accuracy with a Larger Gap

CatBoost achieved a test MAE of $16,528.23, the lowest among the evaluated models. However, its training R^2 of 0.9816 and test R^2 of 0.9041 resulted in a larger generalization gap of 7.75 percentage points.

Although CatBoost delivered excellent average dollar accuracy, XGBoost maintained a smaller train-test gap while achieving higher test R^2 and lower RMSE.

C. Ridge Regression: Underfitting

Ridge Regression recorded a small generalization gap of 0.52 percentage points, but its test R^2 was only 0.8673.

The small gap does not necessarily indicate superior model quality. In this case, the linear model was less capable of capturing the nonlinear relationships and interactions present in property valuation, resulting in underfitting.

D. Random Forest and LightGBM: Overfitting

Random Forest and LightGBM achieved high training R^2 scores but experienced larger declines on the test set.

Random Forest's generalization gap was 10.17 percentage points, while LightGBM's was 11.03 percentage points. These results suggest that both models fitted training-specific patterns more strongly than the leading alternatives.

Overall, the bias-variance analysis reinforced the decision to select XGBoost as the production champion.

7.6 Final Production Champion: XGBoost Regressor

The XGBoost Regressor was selected as the final production model following the comparative evaluation of all five regression algorithms.

The selected model is documented at models/best_model/XGBoost_model.joblib, with an additional copy at models/best_model/XGBoost.joblib.

The decision was based on the following findings.

A. Strong Overall Predictive Performance

XGBoost achieved a test R^2 score of 0.9073, the highest among all evaluated models. This indicates that it explained approximately 90.73% of the variation in sale prices within the held-out test dataset.

B. Lowest RMSE and Dollar-Scale MSE

XGBoost achieved an RMSE of $26,670.54 and a dollar-scale MSE of $711,317,869.42, both the lowest among the evaluated models.

These results were important because larger valuation errors can have a substantial impact on the usefulness of a property price prediction system.

C. Best Generalization Among Nonlinear Models

XGBoost recorded the smallest generalization gap among the nonlinear models, at 5.71 percentage points. This provided evidence that it maintained a better balance between training fit and unseen-data performance than CatBoost, Random Forest, and LightGBM.

D. Suitable for Production Inference

The project documentation reports an inference runtime of less than 3 milliseconds for the XGBoost model in the FastAPI application. This supports its use in an interactive property valuation system that requires quick responses.

E. Balanced Multi-Metric Ranking

XGBoost achieved an overall rank score of 1.25, outperforming CatBoost at 1.75. Although CatBoost achieved a lower MAE and log-scale MSE, XGBoost performed better on R^2, RMSE, dollar-scale MSE, and generalization.

Final decision: XGBoost Regressor was selected as the production champion because it delivered the strongest overall combination of predictive accuracy, control of large dollar-scale errors, generalization performance, and production inference efficiency. CatBoost remained the strongest alternative, particularly when minimizing average absolute dollar error was the primary objective.

8. CONSOLIDATED MASTER DECISION LOG

This section documents the major architectural, mathematical and algorithmic decisions made throughout the project. Each decision includes its rationale and the alternative that was considered or rejected.

Decision ID	Project Phase	Lead Member	Decision Taken	Rationale and Justification	Alternative Considered and Rejected
DEC-01	Formulation	All	Adopt RMSLE and log-domain optimization.	The target is right-skewed, and proportional price errors are important in property valuation.	Standard RMSE optimization on raw prices, which could give excessive influence to expensive properties.
DEC-02	EDA	Venuri	Differentiate structural NA from missing data.	Missing values in features such as PoolQC and GarageType can indicate that a physical feature is absent.	Dropping all columns with more than 50% missing values, which could remove useful information about luxury properties.
DEC-03	EDA	Venuri	Retain high-leverage outliers under log transformation.	Two Edwards properties larger than 4,000 square feet had unusually low sale prices. Log transformation was used to reduce their influence.	Automatically removing these records and potentially losing useful variation and sample size.
DEC-04	Preprocessing	Jayashan	Engineer OverallQual_TotalSF.	Combines overall quality with total enclosed space to represent a potentially nonlinear property valuation relationship.	Using only separate additive quality and area features.
DEC-05	Preprocessing	Jayashan	Replace YearBuilt with HouseAge.	HouseAge represents elapsed time relative to the sale year and directly captures the property's age at sale.	Keeping YearBuilt as the only continuous measure of construction time.
DEC-06	Preprocessing	Jayashan	Correct negative RemodAge values using max(0, age).	One record had a remodeling year later than the recorded sale year, resulting in an age of -1.	Imputing the value with the mean or removing the affected record.
DEC-07	Preprocessing	Jayashan	Drop OutdoorArea.	The subtraction involving LotArea introduced noise without sufficient predictive value.	Retaining every engineered feature regardless of its usefulness.

Decision ID	Project Phase	Lead Member	Decision Taken	Rationale and Justification	Alternative Considered and Rejected
DEC-08	Preprocessing	Jayashan	Prune highly correlated and associated features.	Numerical pairs with absolute Pearson correlation of at least 0.80 and categorical associations around the Cramér's V threshold were inspected for redundancy.	Keeping all strongly redundant features without inspection.
DEC-09	Preprocessing	Jayashan	Separate ordinal encoding from nominal one-hot encoding.	Seven features have meaningful category order, while nominal categories do not have an inherent ranking.	Applying one-hot encoding to all categorical features and potentially creating a much larger feature space.
DEC-10	Preprocessing	Jayashan	Enforce a leakage-free ColumnTransformer pipeline.	Scalers and encoders were fitted using training data and then applied to the test data.	Preprocessing the entire dataset before splitting, which could introduce data leakage.
DEC-11	Training	Teshan	Use five-fold RandomizedSearchCV on the log scale.	Multiple folds provide a more robust basis for hyperparameter selection than one validation split.	Using only a single train-validation split.
DEC-12	Training	Teshan	Compare multiple algorithm families.	Ridge, Random Forest, XGBoost, LightGBM and CatBoost provide different modeling approaches.	Training and evaluating only one algorithm.
DEC-13	Evaluation	Nadeesha	Rank models using MAE, RMSE, log MSE and R^2.	MAE captures absolute dollar error, RMSE penalizes large errors more strongly, log MSE evaluates transformed-target error, and R^2 measures explained variation.	Selecting a model based only on R^2.
DEC-14	Deployment	Nadeesha	Select XGBoost as the production champion.	CatBoost achieved the lowest MAE and log-scale MSE, while XGBoost achieved the highest R^2, the lowest RMSE, and the best overall ranking across the four principal evaluation metrics.	Deploying XGBoost as the selected production champion based on its best overall ranking across the four principal evaluation metrics.
DEC-15	Deployment	Nadeesha	Build an interactive FastAPI application.	The application supports real-time inference, predefined property examples and an interactive valuation interface.	Providing only a command-line prediction script.

9. WEB APPLICATION AND INTERACTIVE DEPLOYMENT

Phase leader: Member 4 - Nadeesha Malshan.

The final model was integrated into a web application developed using FastAPI, Jinja2 and responsive CSS/JavaScript.

The application provides an interface through which users can enter property attributes and receive predicted sale prices.

9.1 Application Architecture

The application is implemented in:

app.py

The source report describes the application as loading the trained XGboost model and preprocessing artifacts during startup.

Reported artifact paths:

- models/best_model/XGBoost_model.joblib

The preprocessing bundle ensures that incoming property information can be transformed using the preprocessing logic expected by the model.

9.2 REST API Endpoints

A. GET /

Purpose: Display the interactive property valuation interface.

This endpoint renders the user interface through which users can enter property characteristics and request a predicted sale price.

B. POST /predict

Purpose: Predict a residential property's sale price.

The reported process is:

1. Accept a JSON payload containing property attributes.
2. Apply the preprocessing bundle.
3. Pass the processed features to the XGBoost model.
4. Convert the log-scale prediction back into a dollar amount using expm1.
5. Return the predicted valuation and the confidence boundaries described in the source document.

The prediction endpoint connects the frontend with the trained machine learning model.

C. GET /presets

Purpose: Provide predefined property examples.

The application includes three property archetypes.

Luxury Executive Villa

- Neighborhood: Northridge Heights.
- Overall quality: 10/10.
- Garage capacity: Three cars.
- Living area: 3,200 square feet.

Suburban Family Residence

- Neighborhood: College Creek.
- Overall quality: 7/10.
- Garage capacity: Two cars.
- Living area: 1,710 square feet.

Historic Starter Cottage

- Neighborhood: Old Town.
- Overall quality: 5/10.
- Garage: Detached.
- Living area: 1,100 square feet.

These presets allow users to explore the application without manually entering every property attribute.

D. GET /health

Purpose: Provide an operational health check.

This endpoint helps determine whether the application is responding to health-check requests.

E. GET /feature-summary

Purpose: Return feature metadata and distributions.

The endpoint provides information about the features used by the application.

9.3 Deployment Workflow

The deployment process follows these steps:

1. Complete model training.
2. Evaluate the five trained models.
3. Select XGBoost as the champion.
4. Save the selected model and preprocessing artifacts.

5. Load the artifacts when the FastAPI application starts.
6. Accept property attributes through the user interface or prediction endpoint.
7. Transform the incoming attributes.
8. Generate a log-scale prediction.
9. Convert the prediction back to the original dollar scale.
10. Return the predicted valuation to the user.

This workflow connects the machine learning pipeline with a practical property valuation application.

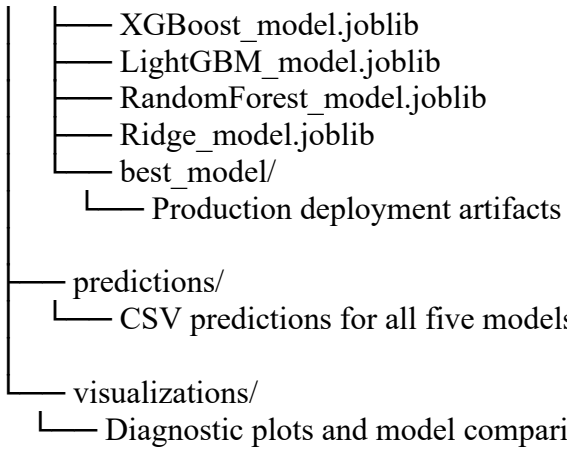
10. CODE WALKTHROUGH AND REPRODUCIBILITY GUIDE

The project repository is organized into separate notebooks, datasets, model artifacts, prediction files and deployment components.

10.1 Project Directory Structure

The following structure summarizes the main project files.

```
ames-housing-price-prediction/
│
├── dataset.csv
│   └── Raw Ames Housing dataset (1,460 rows × 81 columns)
│
├── eda.ipynb
│   └── Member 1: Exploratory Data Analysis and Insight Log
│
├── preprocessV3.ipynb
│   └── Member 2: Data Preprocessing and Feature Engineering v3
│
├── Model_Training.ipynb
│   └── Member 3: Hyperparameter Tuning and Five-Model Training
│
├── Model_Evaluation.ipynb
│   └── Member 4: Model Benchmarking and Champion Selection
│
├── app.py
│   └── Member 4: FastAPI Interactive Web Application
│
├── run.py
│   └── Application startup launcher
│
├── requirements.txt
│   └── Project environment dependencies
│
├── data-new/
│   ├── X_train_preprocessed.csv
│   │   └── Processed training features (1,168 × 91)
│   ├── X_test_preprocessed.csv
│   │   └── Processed test features (292 × 91)
│   ├── y_train.csv
│   │   └── Log-transformed training target (1,168 records)
│   └── y_test.csv
│       └── Log-transformed test target (292 records)
│
├── models/
│   ├── CatBoost_model.joblib
```



10.2 Reproducibility Step-by-Step Instructions

The following procedure describes how to recreate the environment and execute the main project phases on Windows.

Step 1: Create a Python virtual environment

Open a terminal in the project directory and run:

python -m venv .venv

This creates a virtual environment named .venv for the project dependencies.

Step 2: Activate the virtual environment

Run:

.\.venv\Scripts\activate

The virtual environment isolates the project's Python packages from other Python projects on the computer.

Step 3: Install the dependencies

Run:

pip install -r requirements.txt

This installs the packages listed in the project's requirements file.

Step 4: Execute Phase 1 - Exploratory Data Analysis

Open and run:

eda.ipynb

The notebook is used to:

- Inspect feature distributions.
- Examine missingness patterns.
- Identify outliers.

- Explore bivariate relationships.
- Review correlations between features and SalePrice.
- Document the EDA findings.

Step 5: Execute Phase 2 - Preprocessing and Feature Engineering

Open and run:

preprocessV3.ipynb

The notebook performs the following operations:

1. Create engineered features.
2. Select relevant predictors.
3. Reduce redundant features.
4. Create the training and testing partitions.
5. Fit the ColumnTransformer using training data.
6. Transform the numerical, ordinal and nominal features.
7. Apply the log transformation to SalePrice.
8. Export the processed datasets into data-new/.

Step 6: Execute Phase 3 - Model Training

Open and run:

Model_Training.ipynb

This notebook performs the following operations:

1. Execute five-fold RandomizedSearchCV.
2. Tune the five regression models.
3. Fit the selected model configurations.
4. Generate test predictions.
5. Save model artifacts.
6. Generate diagnostic plots.
7. Export the artifacts required by the evaluation phase.

Step 7: Execute Phase 4 - Model Evaluation

Open and run:

Model_Evaluation.ipynb

The notebook performs the following operations:

1. Load the saved predictions.
2. Convert log-scale predictions back to dollar values.

3. Calculate MAE, RMSE, log MSE and R^2.
4. Compare the performance of all five models.
5. Generate model comparison visualizations.
6. Select XGBoostt as the champion model.
7. Save the artifacts required for deployment.

Step 8: Launch the web application

Run:

python app.py

According to the source report, the application can be accessed locally at:

http://127.0.0.1:8000

The application provides the interactive interface for entering property attributes and obtaining a predicted sale price.

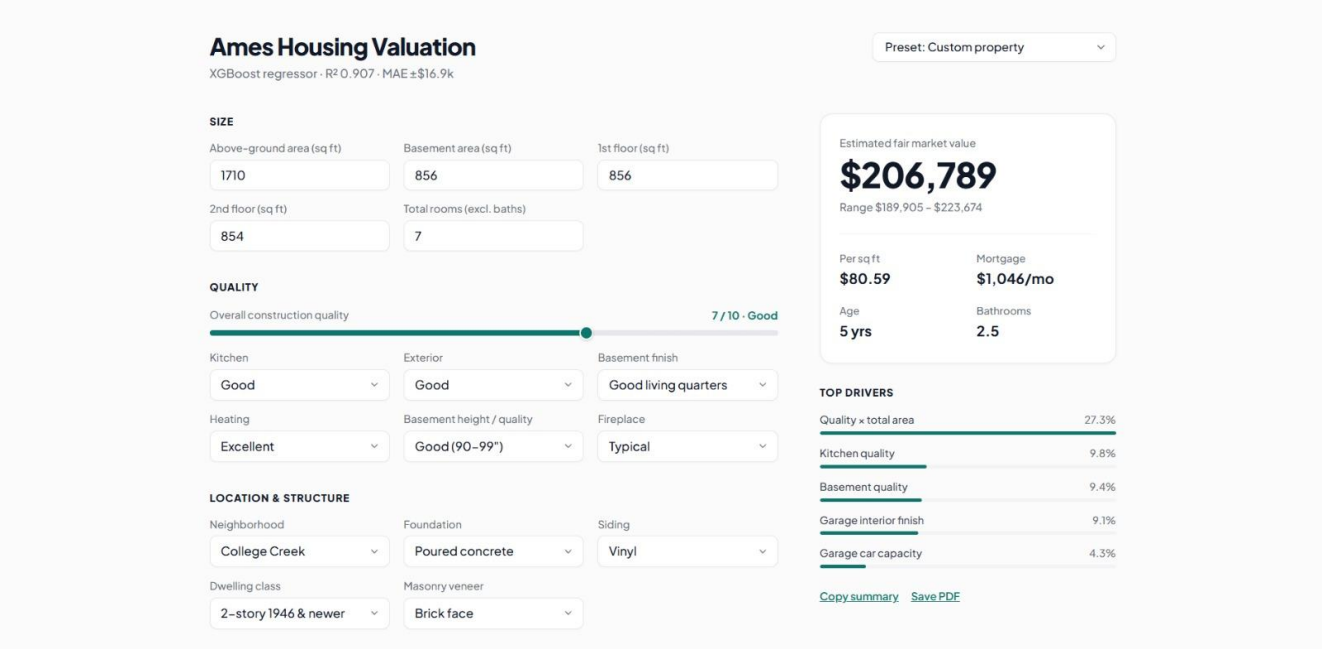


Figure 2 : Application Interface

10.3 Reproducibility Considerations

To reproduce the reported results, the following conditions should be maintained:

- Use the same source dataset.
- Use the same feature-engineering definitions.
- Maintain the same training and testing partition.
- Fit preprocessing transformations using training data only.
- Use the same model hyperparameters.
- Maintain the same target transformation and inverse transformation.
- Evaluate the models using the same test dataset and metrics.
- Ensure the deployment application loads the correct model and preprocessing artifacts.

For exact numerical reproducibility, the random seeds, package versions and execution environment should also be recorded in the project configuration. These details should be verified in the actual notebooks and requirements file.

11. CONCLUSION AND FUTURE WORK

11.1 Key Scientific Findings

A. Compounding nonlinear interactions

The reported strongest engineered predictor was OverallQual_TotalSF, with a Pearson correlation of approximately 0.88.

It combines overall property quality with total enclosed space. The feature was more strongly correlated with SalePrice than either TotalSF, reported at approximately 0.78, or OverallQual, reported at approximately 0.79.

This finding supports the usefulness of engineering features that combine property characteristics rather than relying exclusively on individual raw attributes.

B. Temporal property age features

The project transformed calendar years into relative age measurements:

- HouseAge = YrSold - YearBuilt.
- RemodAge = YrSold - YearRemodAdd.

The source report identified a negative correlation of approximately -0.52 between HouseAge and SalePrice.

These features provide a direct representation of a property's age at the time of sale and the time elapsed since remodeling.

C. Performance of gradient boosting ensembles

CatBoost and XGBoost achieved stronger reported results than Ridge Regression and Random Forest on several evaluation metrics.

The reported R^2 scores were:

- Ridge Regression: 0.8673
- Random Forest: 0.8756
- LightGBM: 0.8589
- CatBoost: 0.9041
- XGBoost: 0.9073

The results demonstrate the value of comparing different regression approaches instead of assuming that one algorithm will always perform best.

D. XGBoost as the Production Champion

XGBoost was selected as the final production champion based on its overall ranking across MAE, RMSE, dollar-scale MSE, and R^2.

Its test-set performance was:

- **Mean Absolute Error (MAE):** $16,884.70
- **Root Mean Squared Error (RMSE):** $26,670.54
- **Mean Squared Error (MSE):** $711,317,869.42
- **Coefficient of Determination (R^2):** 0.9073
- **Overall Average Rank:** 1.25

CatBoost achieved the lowest MAE of $16,528.23 and the lowest reported log-scale MSE of 0.0206. However, XGBoost achieved the best overall average rank across the four principal ranking metrics, making it the selected production model.

The selected XGBoost model was integrated into a FastAPI web application, enabling users to enter property attributes and obtain predicted residential sale prices

11.2 Future Enhancements

A. Stacked Generalization (Super Learner)

A future version could combine the predictions of CatBoost, XGBoost and Ridge Regression using a meta-model.

The proposed approach is to use a Lasso meta-regressor to learn how to combine the predictions from the base models.

The objective would be to determine whether the models capture complementary patterns and whether combining them can improve prediction accuracy.

B. Geospatial and Neighborhood Embeddings

The current feature space represents Neighborhood using one-hot encoding.

A future enhancement could incorporate continuous geographic coordinates, including latitude and longitude, together with distance-to-amenity features.

Potential distance features include proximity to Iowa State University and commercial districts.

These features could provide more detailed information about the geographical relationships between properties and local amenities.

C. Macroeconomic Covariates

The model could be extended by including historical economic indicators for the 2006–2010 transaction period.

Potential variables include:

- Historical mortgage interest rates.
- Inflation indices.
- Local unemployment rates in Ames.

These variables could help represent broader economic conditions that influence residential property prices.

FINAL CONCLUSION

The Ames Housing Price Prediction project demonstrates an end-to-end machine learning workflow, beginning with raw data analysis and ending with an interactive property valuation application.

The team organized the project into four phases: exploratory data analysis, preprocessing and feature engineering, model training and hyperparameter tuning, and model evaluation and deployment.

The preprocessing phase produced 24 selected predictors and a processed feature matrix containing 91 features after transformation. Five regression algorithms were trained and evaluated using five-fold cross-validation and a held-out test dataset containing 292 records.

Following comparative evaluation, XGBoost was selected as the final production champion, achieving an R^2 score of 0.9073, an RMSE of $26,670.54, and an MAE of $16,884.70. Its overall average rank of 1.25 was the best among the evaluated models across MAE, RMSE, dollar-scale MSE, and R^2.

CatBoost also demonstrated competitive performance, achieving the lowest MAE and the lowest reported log-scale MSE. Nevertheless, XGBoost provided the strongest overall performance under the project's selected ranking methodology.

The final model was integrated into a FastAPI web application that supports interactive residential property price prediction. This demonstrates how data analysis, feature engineering, model comparison, evaluation, and deployment can be combined to develop a practical machine learning solution.

Future improvements could investigate stacked generalization, geospatial features, and historical macroeconomic variables. These enhancements would require further experimentation and validation to determine whether they provide measurable improvements over the current production model.